

EchoMind is a private AI assistant. It answers questions from your own documents, listens to your meetings, holds a spoken conversation, and writes finished documents — all on one machine you own. No cloud, no external API calls, no data leaving the building.

The five things it does

Knowledge Chat

Ask questions, get cited answers from your documents.

Live Transcript

Real-time meeting transcription with a fact-checker running alongside.

Conversation

Full spoken dialogue — you talk, it answers out loud.

Document Studio

Generates PDF and PowerPoint deliverables from a brief.

Boardroom

Records multi-speaker sessions, separates speakers, produces a report.

Key facts

Deployment	One NVIDIA DGX Spark (GB10 Grace-Blackwell, 128 GB unified memory). Five Docker containers, one command.
Language model	nvidia/Llama-3.3-70B-Instruct-FP4 served by TensorRT-LLM over an OpenAI-compatible endpoint.
Embeddings	nomic-embed-text (768 dimensions), served locally by Ollama.
Speech	NVIDIA Nemotron streaming ASR for live captions, Parakeet-TDT for the accurate final transcript, Piper for speech output.
Images	SDXL-Turbo running on the same GPU for Document Studio artwork.
Retrieval	FAISS dense vectors + BM25 keyword index, fused and re-ranked by a cross-encoder.
Storage	SQLite plus local disk. Nothing is written outside the machine.
Network	Runs with the cable unplugged. Optional public access via Cloudflare Tunnel behind a login wall.

How to read this document

- Page 2 is the full architecture on one landscape page — start there.
- Pages 3–4 describe every feature: what it does, how it works, why it matters.
- Pages 5–7 are the three pipelines in detail: ingestion, answering, and speech.
- Page 8 is the security and trust model. Page 9 is the code map and operations.

The model named above is the model specified for this deployment; the platform is model-agnostic behind a single endpoint setting.

System Architecture

Every component, every path, one machine

ECHOMIND ENTERPRISE

Architecture Brief v2 · page 2

CLIENTS

browser only — nothing to install

BROWSER — React single-page app

Knowledge Chat · Live Transcript · Conversation · Document Studio · Boardroom
vertical packs: health · law · meetings · retail · bank — each subdomain loads its own theme, persona and knowledge base

ADMIN

users · roles · tenants
audit log · usage

EDGE

TLS termination and routing

NGINX

serves the SPA · /api → backend · /voice → voice service (WebSocket) · TLS

CLOUDFLARE TUNNEL (optional)

outbound-only tunnel · identity gate in front of the app · off by default

APPLICATION

two FastAPI services holding all business logic

BACKEND — FastAPI

documents, retrieval, transcription, generation, identity

Knowledge Chat API

route · retrieve · cite

Ingestion

parse · chunk · embed

Live Transcript

WebSocket · streaming ASR

Silent Assistant

fact-check each paragraph

Boardroom

diarise · analyse · report

Document Studio

PDF · PPTX · images

Export Gateway

scan · redact · release

Auth · RBAC · Audit

sessions · tenants · log

VOICE — FastAPI

the real-time speech loop

VAD + endpointing

when did they stop?

Streaming STT

live partial words

Final STT

accurate transcript

Lead phrase

speaks in < 50 ms

Grounded reply

asks backend RAG

TTS + barge-in

speak · stop on talk

INFERENCE

all model execution, on the local GPU

TensorRT-LLM

Llama-3.3-70B-Instruct
NVFP4 · OpenAI-compatible
answers · analysis · drafting

Ollama

nomic-embed-text
768-dimension vectors
used at upload and at query

Speech models

Nemotron streaming ASR
Parakeet-TDT final ASR
Piper text-to-speech

SDXL-Turbo

on-device image generation
Document Studio artwork
4 steps · up to 1024 px

DATA

local disk only

FAISS

dense vector index
documents + transcripts
flat → IVF at 10k chunks

BM25

exact keyword index
catches names, codes, IDs
rebuilt with the corpus

SQLite

documents · chats · transcripts
users · tenants · audit log
boardroom · docgen jobs

Local disk

uploaded files
generated PDF / PPTX
model weights (baked in)

ONE MACHINE · NVIDIA DGX SPARK GB10 · 128 GB UNIFIED MEMORY · NO EGRESS AT RUNTIME

① browser → nginx over HTTPS

② microphone audio over a WebSocket

③ /api routed to the backend

④ /voice routed to the voice service

⑤ voice asks the backend for a grounded, cited answer

⑥ backend calls the language and embedding models

⑦ voice calls the local speech models

⑧ indexes and records read from local disk

Knowledge Chat RAG cited

- WHAT** Ask a question in plain English and get an answer built only from your uploaded documents and stored meeting transcripts, with a citation on every claim.
- HOW** Hybrid retrieval: a meaning search (FAISS) and an exact-keyword search (BM25) run together, are fused by weighted reciprocal-rank fusion, then a cross-encoder re-reads the top 25 passages against your question and keeps the best 15. The model writes the answer from those passages only.
- WHY** The failure mode of a naive assistant is a confident answer with no source. Here the sources are shown, clickable and traceable to page and section.

Conversational routing intent

- WHAT** The assistant tells the difference between small talk, a refusal, a follow-up and a real question.
- HOW** A prototype-embedding classifier compares your message against learned examples of each intent; greetings and refusals never touch the knowledge base, follow-ups inherit the previous question's context. Rules only as a fast path, semantics as the decision.
- WHY** Without this, saying "hello" pulled four random document extracts and the reply read like a machine. Greetings now get a greeting.

Live Transcript real-time GPU

- WHAT** Speak or play a meeting and see the words appear as they are said, then store the finished transcript into the knowledge base.
- HOW** Audio streams over a WebSocket in 20 ms frames. NVIDIA Nemotron streaming ASR produces partial text continuously; silence boundaries close paragraphs. Transcripts are auto-saved on an interval and become searchable alongside documents.
- WHY** Meetings are where most decisions actually live. Once stored, yesterday's discussion answers today's question.

Silent Assistant fact-check live

- WHAT** While the meeting runs, each finished paragraph is silently checked against your reference material and labelled: Supported, Contradicted, Unverified, Violating or Risky Statement.
- HOW** Every completed paragraph is sent through the same retrieval stack, judged by the model, and only surfaced when confidence is at least 60. Domain rule packs sharpen the check per vertical — drug interactions for clinical, missing disclosures for banking, one-sided clauses for legal.
- WHY** It catches the wrong statement in the room, while it can still be corrected.

Voice Conversation duplex-feel

- WHAT** A spoken conversation. You talk, it answers out loud from the same documents the chat uses, and you can interrupt it mid-sentence.
- HOW** Voice activity detection plus semantic endpointing decide when your turn ends — a complete sentence gets a fast reply, a trailing one waits longer. A short lead phrase starts speaking in under 50 ms while retrieval runs behind it; the grounded answer then streams phrase by phrase into Piper.
- WHY** Turn-based bots feel dead because of the silence after you stop. Filling that gap is what makes it feel like a conversation.

Document Studio PDF PPTX images

- WHAT** Turn a brief into a finished deliverable: clinical notes, contract review memos, meeting minutes, loan suitability reports, whitepapers, pitch decks, PRDs, case studies, brand books and more — 23 templates across five visual themes.
- HOW** The template supplies a section blueprint and writing guidance; the model drafts each section grounded in your source or knowledge base; a renderer produces a real PDF or PowerPoint file. Illustrations are generated on-device by SDXL-Turbo. Custom templates can be uploaded.
- WHY** The output is a file someone can send, not a chat transcript to copy and reformat.

Boardroom multi-speaker

- WHAT** Record a multi-person session, separate who said what, and produce an analysed report you can export.
- HOW** Audio is uploaded in chunks, concatenated, normalised to 16 kHz mono, diarised into speaker turns, linked to the live transcript, then analysed into decisions, commitments and action items.
- WHY** Minutes stop being someone's evening job.

Vertical packs multi-tenant

- WHAT** One deployment presents as five products — Health, Law, Meeting Rooms, Retail and Bank — each on its own subdomain with its own knowledge base, persona, terminology and colour theme.
- HOW** Each pack maps to an isolated namespace. The namespace is applied inside the search itself, not as a filter afterwards, so a query in one pack cannot retrieve another pack's content even in the ranking stage.
- WHY** Sell the same box to a hospital and a law firm without either seeing the other's documents.

Secure Export Gateway offline → online

- WHAT** Before any content leaves the machine, it is scanned for sensitive data and a redacted version is produced.
- HOW** Deterministic detectors — regular expressions plus a Luhn check for card numbers — find national IDs, API keys, private keys, tokens, card numbers, emails, phone numbers and IP addresses, rank them by severity and mask them. No GPU, no network, so it runs air-gapped on every export.
- WHY** "Nothing leaves unchecked" has to be a mechanism, not a policy document.

Pipeline 1 — Getting Documents In

From an uploaded file to a searchable, isolated passage

ECHOMIND ENTERPRISE

Architecture Brief v2 · page 5

Everything the assistant knows arrives through this path. It runs once per document, at upload.



What happens at each step

Extract	Text is pulled out with its structure intact — headings, sections and page numbers are preserved so a citation can point at a real place in the file.
Chunk	The document is cut into passages of roughly 450 words with 120 words of overlap, so a sentence spanning a boundary is never lost. Structured books use chapter-aware chunking with parent/child passages.
Coverage guard	After chunking, the pipeline measures how much of the original text the chunks actually cover. Below 98 %, it discards the clever chunking and falls back to the simple splitter. Content loss is never silent.
Embed	Each passage becomes a 768-number vector capturing its meaning, produced locally by nomic-embed-text.
Index	Passages are filed twice — into FAISS for meaning search and into BM25 for exact words. Both are needed: meaning search finds paraphrases, keyword search finds part numbers and case codes.
Tag	Every passage carries its tenant namespace, document ID, section and page. This tag is what makes isolation and citation possible later.

Design decisions worth knowing

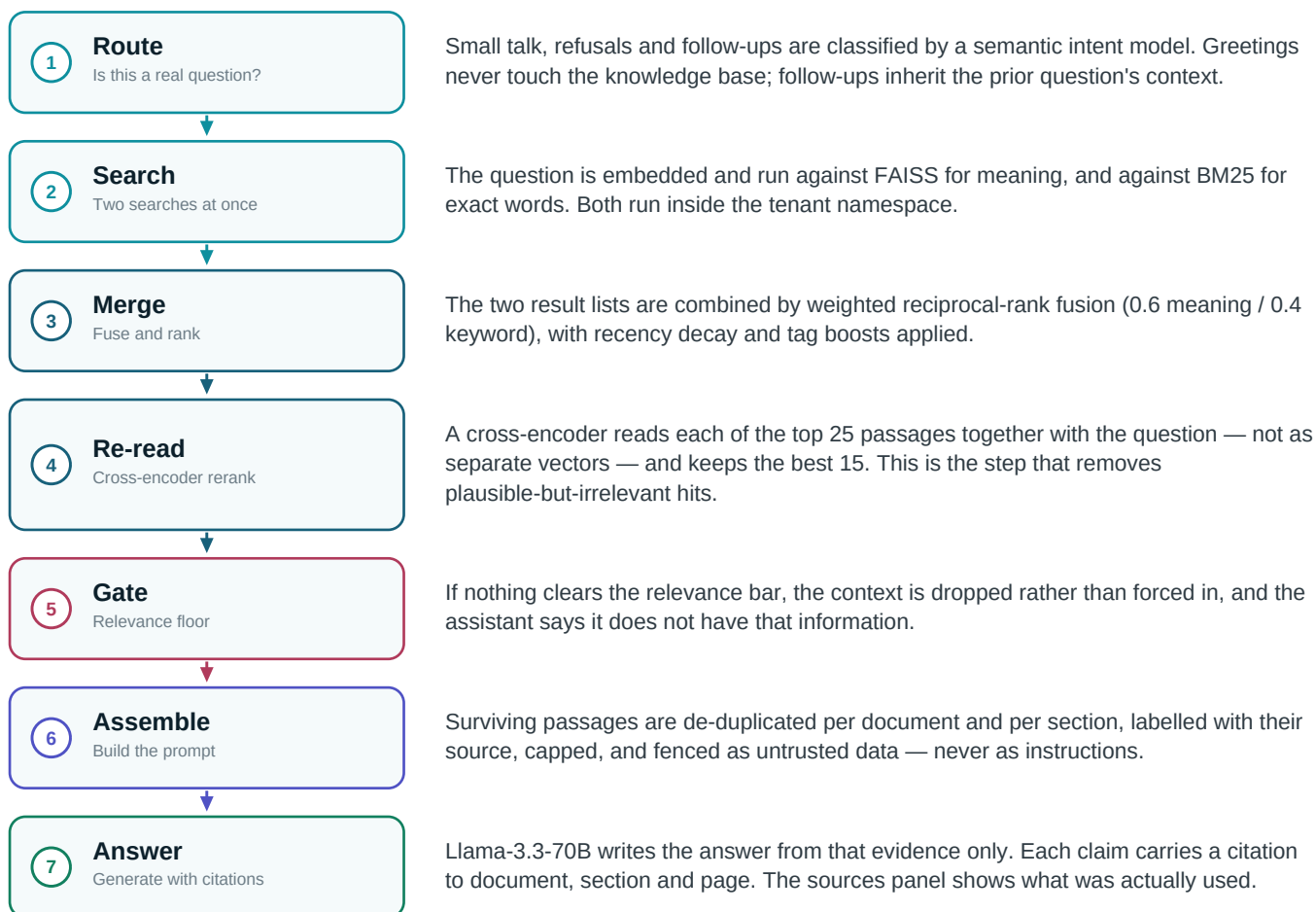
- Two indexes, not one. A single vector index misses exact identifiers; a single keyword index misses paraphrase. Running both and fusing them is measurably better than either.
- Overlap costs storage and buys recall. 120 words of overlap on a 450-word window is the setting that stopped answers being cut in half at chunk boundaries.
- FAISS starts flat and switches to an approximate index above ten thousand chunks — exact search while the corpus is small, scalable search once it is not.
- The namespace is written into the passage at index time. Nothing downstream has to remember to filter.

Pipeline 2 — Turning a Question Into an Answer

Seven stages, every one of them defensible

ECHOMIND ENTERPRISE

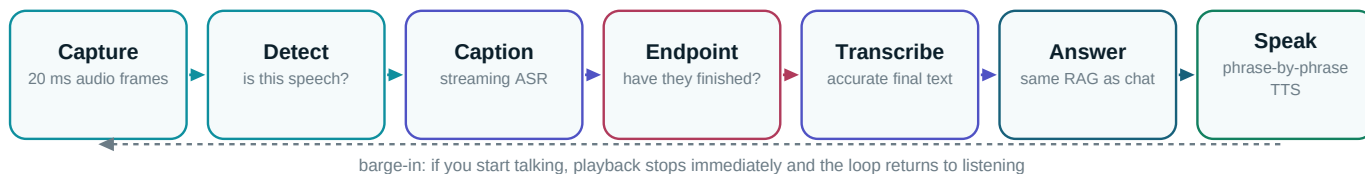
Architecture Brief v2 · page 6



Why the answers can be trusted

- Every claim carries a citation — document, section and page — and the passages used are shown to the reader.
- Retrieved text is fenced as data, never as instructions, so a document cannot hijack the assistant by containing the words "ignore your rules".
- If the documents do not contain the answer, the assistant abstains instead of guessing.
- Tenant isolation is enforced inside the search, not by filtering results afterwards — the wrong tenant's passage never enters the ranking.

The voice service runs a continuous loop. The hard part is not transcription — it is knowing when your turn ended, and not leaving silence while it thinks.



The two parallel tracks

LEAD — starts immediately

A short, context-appropriate filler phrase is chosen by heuristic — no model call — and spoken in under 50 ms. It covers the retrieval time so there is no dead air.

Never carries facts. It cannot be wrong.

GROUNDED — arrives behind it

The real answer runs the full retrieval pipeline against your documents, then streams into speech phrase by phrase as the sentences complete.

Every factual statement comes from here.

Turn-taking, in settings you can actually tune

Complete	550 ms of silence ends your turn when the sentence sounds finished — a fast, natural reply.
Default	700 ms of silence in the ordinary case.
Incomplete	1300 ms when you trail off mid-thought, so a pause to think is not mistaken for the end.
Barge-in	About 160 ms of your speech stops playback — low enough to interrupt easily, high enough to ignore its own echo.
Speech out	Piper synthesises each phrase; edges are faded and controlled pauses inserted (160 ms at sentence ends, 90 ms at clauses) so the joins are inaudible.

Two speech-to-text models run in tandem: a streaming model for live captions, and Parakeet-TDT for the final transcript that actually feeds the language model. Live captions can be approximate; the text the model reasons over cannot.

Five layers

Perimeter

The whole system runs on one machine with no runtime egress. Model weights are baked into the images, so it works with the network cable unplugged. Public access, when enabled, is an outbound-only Cloudflare Tunnel behind an identity gate — no inbound ports opened.

Identity

Username/password sessions with signed tokens, roles and per-user tenant assignment. Admins manage users, see the audit log and usage. The voice WebSocket validates the same session signature before accepting audio.

Tenant isolation

Each vertical pack is a namespace applied inside the retrieval call itself. Both the dense and the keyword path enforce it, including the transcript indexes. The reserved 'default' tenant cannot be used to grant blanket cross-tenant access.

Prompt integrity

Retrieved passages are wrapped and presented to the model as untrusted data. Instructions embedded in an uploaded document are treated as text to be reported, not commands to be obeyed. Reasoning traces are stripped before display.

Egress control

The export gateway scans any content on its way out for national IDs, card numbers (Luhn-verified), API keys, private keys, tokens, emails, phones and IPs, then returns a severity ranking and a masked version.

Failure behaviour — what it does when it is unsure

- No relevant passage clears the gate → the assistant says it does not have that information. It does not fill the gap.
- Chunking loses more than 2 % of a document → the clever path is discarded and the proven splitter runs instead.
- A speech model hits a fatal GPU fault → the health endpoint returns 503, the container exits and restarts with a fresh context. No silent degradation to CPU.
- A subsystem is unavailable → it is reported. The system is built so that a broken component is loud, not quietly wrong.

Auditability

Answers carry citations. Uploads, deletions, logins and exports are written to an audit table. A 52-question golden evaluation suite runs before a release and reports pass rate per vertical, so a retrieval regression is caught by a number rather than by a user.

Code map

backend/app/rag/

index.py
advanced.py
chunking/
intent.py

The answer engine: retrieval, ranking, prompts, personas, gating.

FAISS and BM25 indexes; tenant namespace enforcement lives here.

The seven-stage answer pipeline, end to end.

Document splitting, structure detection and the coverage guard.

Semantic intent classifier — small talk, refusal, real question.

backend/app/transcribe/

Live transcription WebSocket and the Silent Assistant fact-checker.

backend/app/docgen/

Document Studio: templates, themes, PDF and PPTX renderers, image generation.

backend/app/boardroom/

Multi-speaker session capture, diarisation and analysis.

backend/app/core/

Config, database, auth and RBAC, audit log, export gateway.

voice/app/session.py

The whole speech loop: listening, endpointing, lead phrases, barge-in, speaking.

voice/app/adapters/

Speech-to-text, text-to-speech and LLM streaming adapters.

frontend/

React app. packs.ts maps each subdomain to its tenant, persona and theme.

docker-compose.yml

The five services, their models, ports and GPU assignment.

eval/

52-question golden suite plus chunk-coverage and voice end-to-end tests.

The five containers

trtllm

TensorRT-LLM serving
Llama-3.3-70B-Instruct NVFP4
on an OpenAI-compatible
endpoint.

backend

FastAPI — documents, retrieval,
transcription, generation,
identity.

voice

FastAPI — the real-time speech
loop.

ollama

Local embedding model.

frontend

Ngix serving the React app
and routing /api and /voice.

Operations

- One command starts everything: `docker compose up -d`. Public access is a separate profile, off by default.
- Health checks on the backend and voice services detect fatal GPU faults and restart the container with a clean CUDA context.
- Model weights ship inside the images and the runtime is set to offline mode — no download at start-up, no dependency on a registry.
- Data lives in named Docker volumes: the knowledge base, the model cache and the embedding store survive a rebuild.
- Releases are gated on the golden evaluation suite; a drop in pass rate blocks the change.

A note on the language model

EchoMind is model-agnostic. The language model is reached through a single OpenAI-compatible endpoint setting, so it can be changed without touching application code. This document specifies `nvidia/Llama-3.3-70B-Instruct-FP4`, which fits comfortably in the 128 GB unified memory of the GB10.